

Assignment: Has-A Relationship in Java

In Object-Oriented Programming (OOP), a **Has-A relationship** means that one class contains another class as a part of it.

Your task is to write Java programs that show **real-life Has-A relationships** using **Composition** and **Aggregation**.

Problem 1: Car and Engine (Composition)

Think about a car in real life. A car **cannot exist without an engine**.

- Create a class `Engine` with details: `engineNumber`, `horsePower`, and `type` (e.g., Petrol, Diesel, Electric).
- Create another class `Car` with details: `carName`, `model`, and an object of `Engine`.
- Write a method to display the car details along with engine details.
- Create objects and show that a car has an engine.

🔑 Hint: This is **Composition** because a car cannot run without an engine.

Problem 2: Library and Books (Aggregation)

In a library, there are many books. But even if the library is closed, the books still exist.

- Create a class `Book` with details: `title`, `author`, and `ISBN`.
- Create another class `Library` that stores a list of books.
- Add methods to:
 - Add a book
 - Remove a book
 - Display all books in the library
- Create a library and show that a library has many books.

🔑 Hint: This is **Aggregation** because books can exist without the library.

Problem 3: Student and Address (Composition)

A student must have an address. Without an address, student details are incomplete.

- Create a class `Address` with details: `city`, `state`, and `pincode`.
- Create a class `Student` with details: `rollNo`, `name`, and an `Address` object.
- Write a method to display student details along with their address.
- Create a student and show that a student has an address.

☞ Hint: This is **Composition** because an address is part of the student details.

Problem 4: Employee and Department (Aggregation)

In a company, multiple employees work in the same department.

- Create a class `Department` with details: `deptName` and `location`.
- Create a class `Employee` with details: `empId`, `empName`, and a `Department` object.
- Create multiple employees that belong to the same department.
- Display employee details along with their department.

☞ Hint: This is **Aggregation** because a department can exist even if employees leave.

Problem 5: Bank and Accounts (Aggregation)

A bank manages different accounts. But an account can exist even if the bank closes.

- Create a class `Account` with details: `accountNo`, `balance`, and `type` (Savings/Current).
- Create a class `Bank` that has a list of accounts.
- Add methods to:
 - Open a new account
 - Deposit money
 - Withdraw money
 - Display account details
- Show that a bank has many accounts.

☞ Hint: This is **Aggregation** because accounts can exist outside of the bank.